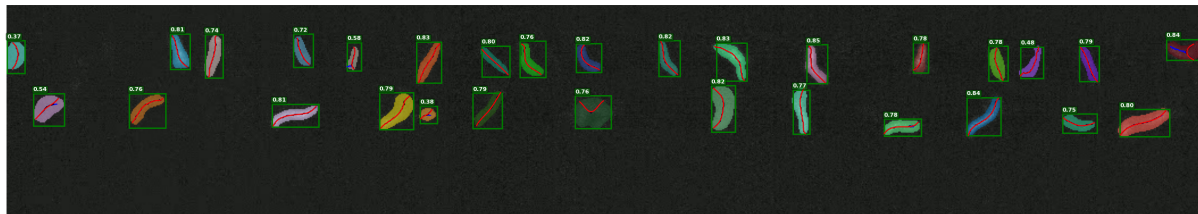


CilialO User Guide

CilialO is a state-of-the-art ML-based quantification methodology for cilia morphodynamics. This repository contains a complete multi-modal pipeline for quantifying ciliary motion from confocal microscopy videos. This workflow relies on single-shot cilia detection using a fine-tuned YOLO v11 model which identifies bounding box regions of interest (ROI) to pass to Meta's Segment Anything (SAM) vision transformer models, removing the need for computationally expensive iterative masking and tracking. Unlike prior work which use traditional computer vision approaches like binary thresholding and Canny detection, CilialO identifies where cilia are, enabling high quality masks even when cilia overlap. The goal of this project is to enable fast, robust, automated analysis of cilia dynamics overtime across diverse experimental conditions. CilialO can detect subtle changes in cilia morphodynamics that may not be detected by the human eye, but are pivotal to developing our understanding of ciliary function overall.



Raw Frame: 4-day-post-fertilization Zebrafish monocilia raw confocal image.



CilialO Frame: High quality detection, segmentation, skeletonization, and quantification of cilia.

Table of Contents

- [CilialO User Guide](#)
 - [Table of Contents](#)
 - [CilialO's Core Pipeline](#)
 - [Directory Structure](#)
 - [0. System Requirements](#)
 - [Hardware](#)
 - [Software](#)
 - [Operating System & Environment](#)
 - [1. Acquiring the Software](#)
 - [Option A: Stable Release \(Zenodo\)](#)
 - [Option B: Graphical Method \(GitHub Desktop\)](#)
 - [Option C: Command Line Method \(Terminal\)](#)
 - **NOTE: IF INSTALLING FROM ANYWHERE EXCEPT ZENODO, YOU NEED TO DOWNLOAD SAM AS WELL**
 - [Downloading SAM for GitHub](#)

- 2. Installation of Requirements
- 3. Creating an Experiment Folder
 - Naming Rule (In general)
 - Note on Flexibility
- 4. Preparing Data and Metadata
- 5. Running CiliaIO
 - Execution Commands
 - Dorsal/Ventral Static Y-Threshold Mode
 - Batch List Mode
 - Manual Calibration for Metadata
 - Another Real-World Example Usage
 - All Parameter Definitions
 - Required Parameters
 - Dorsal Ventral Y-Axis Thresholding Parameters
 - Calibration and Metadata
 - Detection and Tracking
 - Skeletonization and Motility
 - Visualization and Debugging
 - CiliaIO Outputs (per video file)
- 6. Troubleshooting and Common Problems
 - Hardware & Environment Errors
 - Data & Folder Errors
 - Terminal Navigation Cheatsheet

CiliaIO's Core Pipeline

1. **Detection**

Cilia are detected frame-by-frame using a fine-tuned YOLOv11m model from Ultralytics.

2. **Tracking**

Object tracks are initialized and maintained using ByteTrack with additional logic for center-distance matching and overlap-based ID merging. Fallback heuristics handle missed detections and ensure temporal ID consistency across frames.

3. **Segmentation** Boxes are segmented using the Segment-Anything Model (SAM).

4. **Skeletonization**

Segmentation masks are skeletonized using the morphological thinning algorithm, which iteratively removes pixels on each side of the mask until one pixel remains.

5. **Quantification**

Tracked trajectories are analyzed to extract per-cilium motion features, including spatial amplitude, beating frequency, beating regularity, centroid frequency, and more. Summary statistics are aggregated across videos for downstream analysis.

6. **Visualization**

The pipeline supports trajectory overlays, segmentation mask overlays on the raw confocal videos, skeleton tracking, and bounding box tracking.

Directory Structure

The following text details the directory structure of all files found in CiliaIO. These are not instructions, but instead serve as a reference for every single file included in CiliaIO.

- **data/**: This is where the data goes
- **sam/**: If coming from github, download the SAM ViT-H checkpoint 4b8939.pth file from Meta. See the section labeled: "Downloading SAM for GitHub" above for more details.
- **src/**: Contains all of the source file pertaining to CiliaIO'
 - **cilia_io.py**: The top main file for the entire repository.
 - **ml_helpers.py**: Helper functions used by CiliaIO for machine learning.
 - **quantification_help.py**: Helper functions used by CiliaIO for quantification.
 - **benchmarking_cilia_tools.py**: Benchmarking cilia_io versus binary thresholding, canny thresholding, and pixel-based frequency segmentation
- **src/analysis/**: Contains all of the source files pertaining to downstream analysis with cilia_io
 - **analyze_csv_data_violin.py**: Uses the cilia_io output CSVs to generate Welch t-tests, 3D graphics, Cohen's power analysis, and violin plots.
 - **analyze_k_fold_cross_val_plots.py**: Contains utilities for coalescing k-fold cross validation plots together
 - **benchmark_segment_anything.py**: Benchmarks zero-shot segment-anything on manual labeled data.
 - **compare_csv_for_accuracy.py**: Script used for calculating the MAPE between manual and CiliaIO quantification outputs.
 - **visualize_skeleton_over_time.py**: Script to take skeleton pickles out of cilia_io output and produce .PNG temporal waveform plots
- **src/dataset_creation**: Create datasets for YOLO fine-tuning and k-fold cross validation
 - **create_yolo_training_dataset.py**: From raw label-studio projects, generates the combined train, validation, and test splits for the fine-tuning of the YOLO model.
 - **create_k_fold_cross_val_dataset.py**: Generate 3-fold stratified cross validation dataset for YOLO validation analysis
- **yolo/**: Contains the original Ultralytics YOLOv11m model, the train and test scripts, as well as the final, fine-tuned model.
 - **configs.yaml**: YAML file for configuring the fine-tuning of the YOLO model.
 - **save_trainig_curves.py**: Saves training curves from the YOLO training flow as .csv files
 - **yolo_test.py**: Runs the test inference on the trained best YOLO model.
 - **yolo_train.py**: Runs training and validation curves for 300 epochs on the dataset.
 - **yolo11m.pt**: The actual pre-trained YOLOv11m model from Ultralytics.
 - **k_fold_cross_val.py**: Runs k_fold cross validation using the dataset created by `'../src/dataset_creation/create_k_fold_cross_val_dataset.py'`

0. System Requirements

Before you begin, ensure your workstation meets the following criteria:

Hardware

- **GPU (Required):** This software uses AI models that require high-performance graphics processing to function. Specifically, they call Torch libraries and use large ML models that typically require a GPU
 - **Windows/Linux:** Requires a dedicated **NVIDIA GPU**.
 - **Mac:** Supports **Apple Silicon (M1, M2, M3 chips)** using the integrated GPU via Apple's Metal framework.
- **RAM:** At least 16GB is recommended for handling large **.TIF** stacks without system lag.

Software

- **Python 3.10 or higher:** The programming language used to run the scripts. (The code was specifically tested in Python 3.10.12)
- **Drivers:**
 - **Windows/Linux:** CUDA Toolkit (compatible with your NVIDIA GPU).
 - **Mac:** Metal Performance Shaders (MPS), which is natively supported on macOS for Apple Silicon.

Operating System & Environment

- **Windows:** The code has **not** been tested in the native Windows environment. You must use **WSL2 (Windows Subsystem for Linux)** to run the analysis.
- **Mac / Linux:** Standard Terminal is sufficient. The scripts are fully tested and supported on macOS and Linux distributions.

1. Acquiring the Software

You can obtain the CiliaO files by downloading a stable release for publication citation (Zenodo) or by syncing the live development code (GitHub).

Option A: Stable Release (Zenodo)

Best for citing in papers or ensuring you are using the exact version used in a specific study.

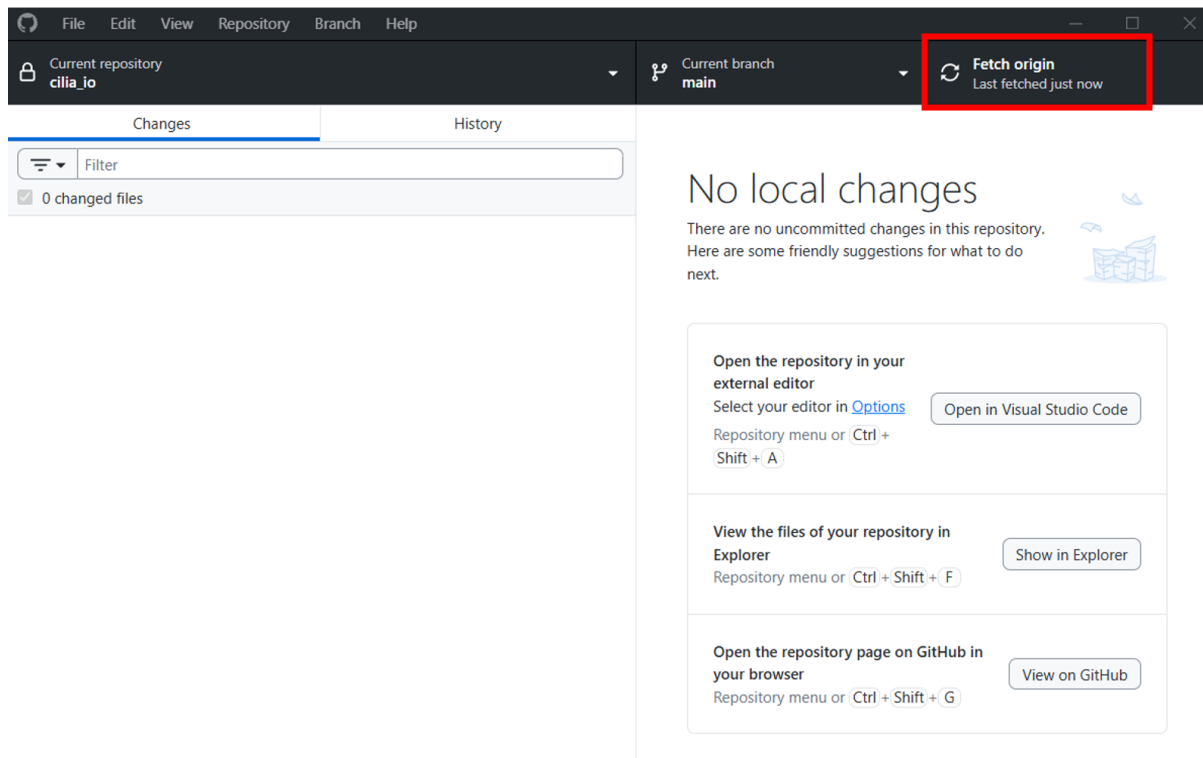
1. Visit the [CiliaO Zenodo repository](#).
2. Download the **Source code (zip)** file from the latest version.
3. Extract the zip folder to your desired location on your computer. *Note: This version is a "snapshot" and will not receive automatic updates.*



Option B: Graphical Method (GitHub Desktop)

Best for users who prefer a visual interface and want to keep the software updated with the latest bug fixes. This works on **Windows and Mac**.

1. **Download:** Go to desktop.github.com and install the app.
2. **Clone:** Click **File > Clone Repository**, select the **URL** tab, and paste the URL for **cilia_io** (https://github.com/chekfung/cilia_io.git).
3. **Update:** To get the latest code, open the app and click **Pull/Fetch Origin**. This "refreshes" your local folder with the newest lab updates.



Option C: Command Line Method (Terminal)

Best for users comfortable with the command line (Windows/Mac/Linux) who already have Git installed.

1. Open your terminal (Ubuntu on Windows; Terminal on Mac/Linux).
2. Run the following command:

```
git clone https://github.com/chekfung/cilia_io.git
```

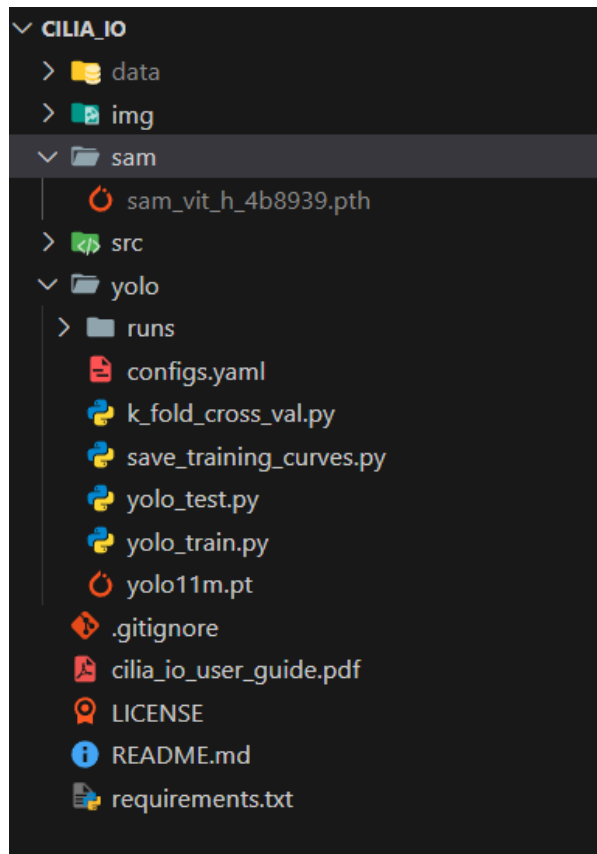
NOTE: IF INSTALLING FROM ANYWHERE EXCEPT ZENODO, YOU NEED TO DOWNLOAD SAM AS WELL

Downloading SAM for GitHub

The Segment-Anything Model is available at this link:

https://dl.fbaipublicfiles.com/segment_anything/sam_vit_h_4b8939.pth

Download the model and then inside of the **cilia_io** repository, create a new folder titled **sam** in the root directory and move the model into that folder. The fine-tuned YOLO model is already installed in both Zenodo and GitHub, but the SAM model is too large. An example of the file structure is shown below.



2. Installation of Requirements

See Requirements.txt for the most up to date dependencies from Python. All results were run with Python 3.10

To install the requirements, we recommend a virtual environment to keep your Python package installation different than the ones required by CiliaIO. You can do this as follows. I would also recommend copy and pasting each line separately so that we can assess the errors independently.

Also, if the terminal is unfamiliar, I have provided a small cheatsheet at the end of the user guide on navigating the terminal and understanding some key commands.

```
cd cilia_io
```

```
python3 -m venv cilia_io_venv
```

```
source cilia_io_venv/bin/activate # On Windows use: .venv\Scripts\activate
```

```
pip install --upgrade pip
```

```
pip install -r requirements.txt
```

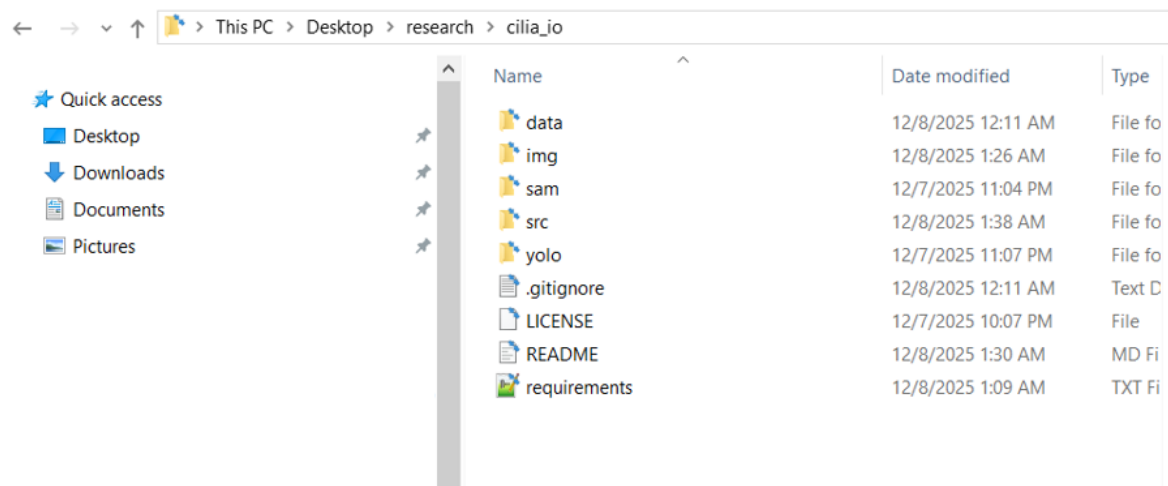
Note: This will take some time because it installs all the PyTorch and CUDA binaries. ~15-20 minutes

3. Creating an Experiment Folder

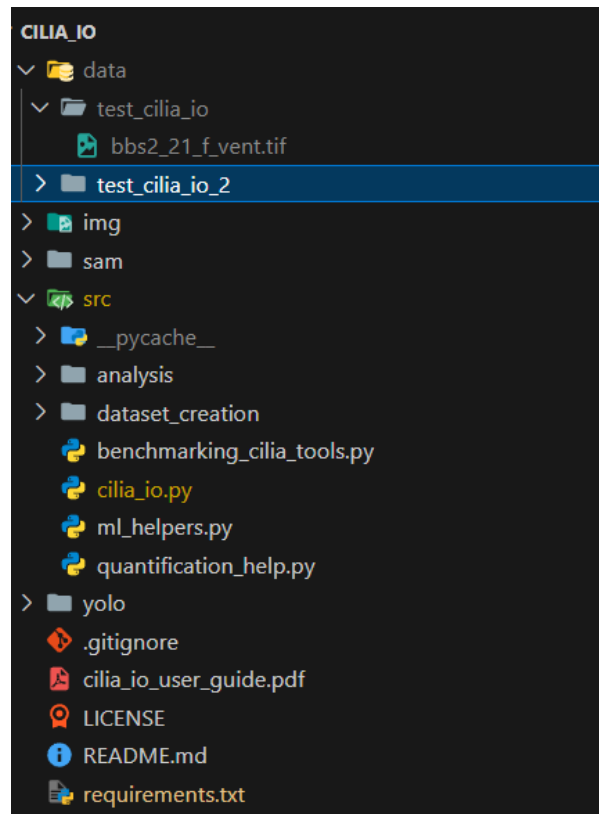
While CiliaIO is flexible and can be pointed to any location on your computer, keeping your data organized within the project directory is recommended for simplicity.

1. **Open the `cilia_io` folder:** Navigate to where you downloaded or cloned the repository.
2. **Access the Data Directory:** Open the folder named `data/`. If it does not exist, create a new folder and name it `data`.
3. **Create your Experiment Folder:** Inside `data/`, create a new sub-folder specifically for your current batch of videos (.TIF)
4. **Move .TIF Files into Folder:** Inside `data/FOLDER_NAME`, insert your .TIF files.

Below is an example of the file structure of CiliaIO. The data folder is in the main directory of `cilia_io`.



If we dive into the data folder, I created a subfolder, in this case, called `test_cilia_io`. Inside, there is a TIF file named `bbs2_21_f_vent.tif`. There is another folder, called `test_cilia_io_2` which can contain other .TIF files that I want to analyze in a separate call to CiliaIO.



Naming Rule (In general)

You must use underscores instead of spaces (e.g., `2026_04_30_cilia_test`). The terminal cannot easily read folder names that contain spaces.

Note on Flexibility

If you prefer to keep your data on an external hard drive or a different local directory, you can do so. You will simply need to provide the **full file path** to that location when you run the script in the Terminal later. Using the internal `data/` folder is the easiest way to avoid path errors, but not required.

4. Preparing Data and Metadata

For this tutorial, we will put everything into the data folder. Inside of the data folder, create

1. **Copy Videos:** Place your raw `.TIF` files into the folder you just created.

5. Running CiliaIO

Note: If you already installed everything, you do not need to repeat the steps above. START HERE!

The pipeline processes all video files within a specified directory. Output results, including processed videos and data files, are saved directly into the source directory.

CiliaIO will go through every single `.tif` file inside of the folder in `input-dir`. This can then be used in the `analyze_csv_data_violin.py` scripts in order to join all the `.csv` files together and perform statistical analysis.

NOTE: To change CiliaIO to run file formats other than .TIF, convert to .TIF first.

1. Open your virtual environment from the previous steps by finding the folder where you installed it and using this command in a terminal. In our case, in this tutorial, we installed the virtual environment in the cilia_io directory:

```
source cilia_io_venv/bin/activate
```

- Quick tip: The most immediate sign is the appearance of the environment name in parentheses at the start of your command line prompt.
 - Active: (cilia_io_venv) user@workstation:~/cilia_io/src\$
 - Inactive: user@workstation:~/cilia_io/src\$
2. Navigate from there to the cilia_io/src directory.

```
cd src
```

Execution Commands

Below are some example commands to run cilia_io with different parameters. These are examples, and parameters can be combined into long command line arguments.

Dorsal/Ventral Static Y-Threshold Mode

This is the default mode. It applies a single horizontal Y-axis coordinate to every video in the folder to define the dorsal and ventral regions. Notably, the "/path/to/videos_directory" is a PLACEHOLDER where you will need to insert your directory that you want CiliaIO to process.

```
python cilia_io.py --input-dir "/path/to/videos_directory"
```

From our previous example using our folder created in data for cilia_io, we would perform the following command:

```
python cilia_io.py --input-dir ../data/test_cilia_io
```

Quick explanation: In the terminal, .. is a shorthand command that means "go up one level" to the parent directory.

Because you are running the software from the src/ folder, but your videos are stored in the data/ folder, you must tell the computer how to navigate between them.

When you type ../data/test_cilia_io, you are giving the terminal a "relative path":

- `..`: Exit the current `src/` folder and move up to the main `cilia_io/` project folder.
- `/data`: From that main folder, look down into the `data/` directory.
- `/test_cilia_io`: Enter the specific folder containing your videos.

Batch List Mode

To apply specific, unique thresholds to videos within the same folder, disable the static toggle and provide a list. The thresholds are applied to files in alphabetical order. The command below applies a Y-axis threshold of 85, 72, and 90 to the first three videos, respectively. This is particularly useful because different videos may occur on different microscopes, have different scaling, etc.

```
python cilia_io.py --input-dir "/path/to/videos_directory" --no-static-dv --dv-threshold-list 85 72 90
```

Manual Calibration for Metadata

If metadata extraction fails due to the specific microscope or corrupted metadata, you must manually provide the temporal and spatial scales (fps and number of microns in a pixel). The below shows an example where we disable the default metadata search, and instead provide the video-fps and pixel-to-micron conversions ourselves.

```
python cilia_io.py --input-dir "/path/to/videos_directory" --no-metadata-search --video-fps 49.88 --pixel-to-micron 0.107
```

Another Real-World Example Usage

This is an example in which we disable the random colors for the cilia output video display, and instead make them based on the base dorsal ventral coloring scheme (can be adjusted in the source code under `DORSAL_COLOR` and `VENTRAL_COLOR`). Then, the output video fps is set to save at 1 FPS, with a minimum number of consecutively tracked frames of 1 to examine motility. The static dorsal ventral threshold has been turned off as well, and set to `Y=80` for the first video, and 85 for the second. This is displayed by the blue line in the output videos. Early stop has also been turned on as a way to examine the first 5 frames to make sure that the dorsal ventral threshold is set correctly.

```
python3 cilia_io.py --input-dir "/path/to/videos_directory" --no-random-colors --output-fps 1 --min-motility 1 --no-static-dv --dv-threshold-list 80 85 --early-stop
```

All Parameter Definitions

These parameter definitions are also available using this command line function for reference:

```
python cilia_io.py --help
```

Required Parameters

- `--input-dir`: Path to the directory containing source videos.

Dorsal Ventral Y-Axis Thresholding Parameters

- `--static-dv` | `--no-static-dv`: Toggles between using a single threshold for the entire batch or a sequence of thresholds from the list. Default is to use one static-dv for all video files
- `--dv-threshold`: Integer value representing the Y-pixel coordinate for the horizontal cut. Default is 80.
- `--dv-threshold-list`: A space-separated list of integers used when static-dv is disabled.

Calibration and Metadata

- `--metadata-search` | `--no-metadata-search`: Toggles automated extraction of FPS and scale from file headers. Default is to search for metadata if not explicitly specified.
- `--video-fps`: Value for frames per second. Required if metadata-search is disabled.
- `--pixel-to-micron`: Value for spatial conversion (AKA. How big is a pixel, in microns?). Required if metadata-search is disabled.

Detection and Tracking

- `--matching-conf`: YOLO confidence threshold for point identification. Default is 0.4.
- `--max-dist`: Maximum pixel distance allowed for matching IDs between frames. Default is 30.
- `--max-age`: Number of frames to retain an unmatched ID before dropping it. Default is 5.
- `--post-iou-thresh`: Intersection over Union (IOU) threshold used for bounding box matching. Default is 0.5.

Skeletonization and Motility

- `--interpolation-pts`: Number of nodes used for spline resolution during skeletonization. Default is 100.
- `--min-motility`: Minimum consecutive frames required to calculate motility metrics. Default is 42.

Visualization and Debugging

- `--output-fps`: Playback speed for the generated result videos. Default is 5.
- `--random-colors` | `--no-random-colors`: Toggles between unique track colors or dorsal/ventral specific coloring. Default is random colors if not specified.
- `--early-stop`: If enabled, the script processes only the initial frames for debugging purposes. Default is False.

CiliaIO Outputs (per video file)

1. Converted mp4 file: `{file_name}.mp4`
2. Converted ciliaIO annotated output: `{file_name}_5fps_ciliaio_output.mp4`
3. Converted ciliaIO annotated output with original fps: `{file_name}_original_fps_output.mp4`
4. Center of Box .PKL: `{file_name}_box_center_points.pkl`

- This is maintained as a dictionary with key (cilia_id) and is a list of tuples consisting of (frame_id, (x_i,y_i)) for the box centroid
- 5. Center of Box Tracking Cilia Quantification .CSV: *{file_name}_quant_box_center.csv*
- 6. Center of Skeleton .PKL: *{file_name}_skel_center_points.pkl*
 - This is maintained as a dictionary with key (cilia_id) and is a list of tuples consisting of (frame_id, (x_i,y_i)) for the skeleton midline
- 7. Center of Skeleton Tracking Cilia Quantification .CSV: *{file_name}_quant_skel_center.csv*
- 8. Full cilia skeleton .PKL file: *{filename}_full_skeletons.pkl*
 - This is maintained as a dictionary of dictionaries. First indexed by cilia_id, then by frame_id, and then the 2D matrix where each row represents the an (x,y) coordinate of one point on the skeleton. The skeleton is ordered, meaning that when incrementing the row index, it represents the next point on the skeleton.

6. Troubleshooting and Common Problems

Even with a perfect setup, you may encounter errors in the terminal. Most issues in CiliaIO are related to file naming, folder paths, or your computer's hardware.

Hardware & Environment Errors

Error Message	What it Means	The Fix
python3: command not found	Python is either not installed or not "mapped" in your terminal.	Windows/WSL: Type <code>sudo apt update && sudo apt install python3</code> . Mac: Install Python via python.org .
CUDA Out of Memory	Your NVIDIA GPU does not have enough memory to process the current video.	1. Close Chrome, Zoom, or other GPU-heavy apps. 2. Try processing a smaller video clip.
ModuleNotFoundError	A required software library is missing.	Run <code>pip install -r requirements.txt</code> again from the main project folder.
NVIDIA-SMI has failed	The terminal cannot talk to your GPU.	Windows: Update your NVIDIA drivers on the Windows side. Mac: Ensure you are on a Mac with an M1/M2/M3 chip.

Data & Folder Errors

Error Message	What it Means	The Fix
FileNotFoundError	The computer cannot find your data folder or metadata file.	1. Check for typos (Linux is case-sensitive). 2. Ensure you are in the <code>src</code> folder. 3. Ensure your folder name has no spaces .
IndexError or KeyError	There is a mismatch or typo inside your <code>metadata.csv</code> .	1. Check that the "filename" column matches your <code>.TIF</code> file names exactly. 2. Ensure there are no empty rows at the bottom of the spreadsheet.

Error Message	What it Means	The Fix
Permission Denied	The computer is blocked from accessing a file.	Make sure the video file isn't currently open in Excel, ImageJ, or a video player.

Terminal Navigation Cheatsheet

Command	Meaning	Description
pwd	Print W orking D irectory	Displays the exact path of the folder you are currently in.
ls	L ist	Shows all files and folders inside your current directory.
cd <folder>	Change D irectory	Moves you into a specific folder (e.g., <code>cd src</code>).
cd ..	Change D irectory ..	Moves you up one level to the "Parent" folder.
cd ~	Change to H ome	Returns you immediately to your primary user home directory.